Lesson: Arrays

CSE 4107 : Structured Programming I

Shahriar Ivan

One-Dimensional Arrays

- An ***array*** is a data structure containing a number of data values of same type
- These values (***elements***) can be individually selected by their positions
- Arranged one after another in a single row



- Declaration: specify the *type* and number of items

```
int a[6];
```

One-Dimensional Arrays

- The ***elements*** may be of any type; the ***length*** of the array can be any (integer) constant expression
- Using a macro to define the length of an array is an excellent practice:

```
#define N 10

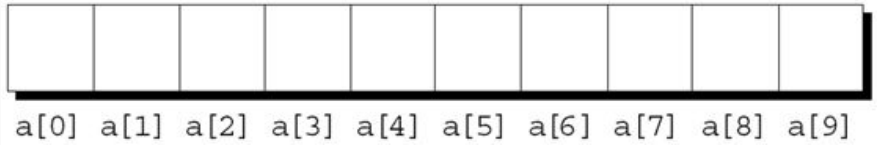
…

int a[N];
```

One-Dimensional Arrays

Array Subscripting

- To access an array element, write the array name followed by an integer value in square brackets.
- This is referred to as ***subscripting*** or ***indexing*** the array.
- The elements of an array of length *n* are indexed from 0 to *n* – 1.
- If a is an array of length 10, its elements are designated by a[0], a[1], …, a[9]:



One-Dimensional Arrays

Array Subscripting

- Expressions of the form a[i] are lvalues, so they can be used in the same way as ordinary variables:

```
a[0] = 1;
printf("%d\n", a[5]);
++a[0];
```

- In general, if an array contains elements of type *T*, then each element of the array is treated as if it were a variable of type *T*

One-Dimensional Arrays

Array Subscripting

- Many programs contain for loops whose job is to perform some operation on every element in an array.
- Examples of typical operations on an array a of length N:

```
for (i = 0; i < N; i++)
      a[i] = 0;                 /* clears a */

for (i = 0; i < N; i++)
      scanf("%d", &a[i]);       /* reads data into a */

for (i = 0; i < N; i++)
      sum += a[i];              /* sums the elements of a */
```

One-Dimensional Arrays

Array Subscripting

- C doesn't require that subscript bounds be checked; if a subscript goes out of range, the program's behavior is undefined
- A common mistake: forgetting that an array with *n* elements is indexed from 0 to *n* − 1, not 1 to *n*:

```
int a[10], i;

for (i = 1; i <= 10; i++)
        a[i] = 0;
```

  - With some compilers, this innocent-looking for statement causes an infinite loop

One-Dimensional Arrays

Array Subscripting

- An array subscript may be any integer expression:

```
a[i+j*10] = 0;
```

- The expression can even have side effects:

```
i = 0;
while (i < N){
   a[i++] = 0;
}
```

One-Dimensional Arrays

Array Subscripting

- Be careful when an array subscript has a side effect:

```
i = 0;
while (i < N)
        a[i] = b[i++];
```

- The expression a[i] = b[i++] accesses the value of i and also modifies i, causing undefined behavior
- The problem can be avoided by removing the increment from the subscript:

```
for (i = 0; i < N; i++)
        a[i] = b[i];
```

Programming Task

Reverse a Series of Numbers

- The program prompts the user to enter a series of numbers, then writes the numbers in reverse order:

  Enter 10 numbers: 34 82 49 102 7 94 23 11 50 31

  In reverse order: 31 50 11 23 94 7 102 49 82 34

- The program stores the numbers in an array as they're read, then goes through the array backwards, printing the elements one by one

One-Dimensional Arrays

Array Initialization

- Can be given initial value at the time of declaration
- Most common ***array initializer*** -> List of constants enclosed in braces, separated by comma

```
int a[7] = {2, 4, 4, 1, 1, 3, 9}
```

- Initialization can be shorter than array

```
int a[7] = {2, 4, 4}        // rest of the uninitialized values are initialized with 0s
```

- Initialize array with all zeros:

```
int a[7] = {0}        // a: {0, 0, 0, 0, 0, 0, 0}
```

One-Dimensional Arrays

Array Initialization

- There's a single 0 inside the braces because it's illegal for an initializer to be completely empty.
- It's also illegal for an initializer to be longer than the array it initializes
- If an initializer is present, the length of the array may be omitted:

```
int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

- The compiler uses the *length* of initializer to determine how long the array is

One-Dimensional Arrays

Designated Initializers (C99)

- Used to initialize few specific elements of array

```
arr[10] = {[6] = 48, [8] = 7, [2] = 29};       // rest are set to 0
```

- Order in which elements are listed does not matter
- Each number in bracket -> Designator
  - Must be a constant expression
- If length is omitted, compiler deduces array length from the largest designator

```
int b[] = {[5] = 10, [23] = 13, [11] = 36};   // an array with 24 elements
```

Programming Task

Checking a Number for Repeated Digits

- The program checks whether any of the digits in a number appear more than once
- After the user enters a number, the program prints either Repeated digit or No repeated digit:

```
Enter a number: 28212

Repeated digit
```

- The number 28212 has a repeated digit (2); a number like 9357 doesn't

Programming Task

Checking a Number for Repeated Digits

- The program uses an array of 10 Boolean values to keep track of which digits appear in a number
- Initially, every element of the *digit_seen* array is ***false***
- When given a number *n*, the program examines *n*'s digits one at a time, storing the current digit in a variable named digit
  - If `digit_seen[digit]` is ***true***, then digit appears at least twice in ***n***
  - If `digit_seen[digit]` is ***false***, then digit has not been seen before, so the program sets `digit_seen[digit]` to true and keeps going

One-Dimensional Arrays

Using the sizeof Operator with Arrays

- Can be used to determine the size of array in bytes

```
int arr[10];

printf("%d\n", sizeof(arr));  // Prints "40" assuming int requires 4 bytes
```

- Find length of array by dividing by the size of single element

```
printf("%d", sizeof(arr)/sizeof(arr[0]));
```

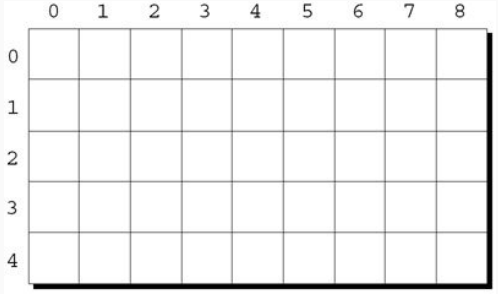
- Easier to modify if size of array changes later
- Better to typecast to int for comparison

Multi-Dimensional Arrays

- An array may have any number of dimensions
- The following declaration creates a *two-dimensional* array (a *matrix,* in mathematical terminology):

```
int m[5][9];
```

- ***m*** has 5 rows and 9 columns. Both rows and columns are indexed from 0:

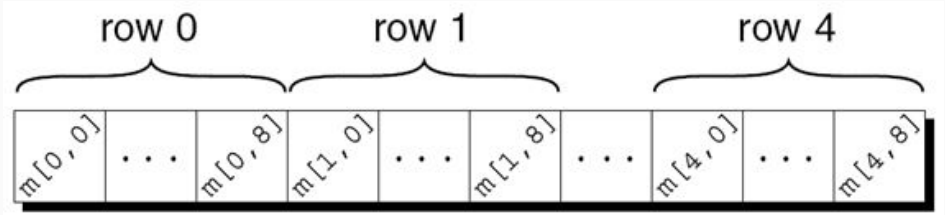


Multi-Dimensional Arrays

- To access the element of m in *row* i, *column* j, we must write m[i][j]
- The expression m[i] designates *row* i of m, and m[i][j] then selects element j in this *row*
- Resist the temptation to write m[i,j] instead of m[i][j]
- C treats the ***comma*** as an operator in this context, so m[i,j] is the same as m[j]

Multi-Dimensional Arrays

- Although we visualize *two-dimensional* arrays as tables, that's not the way they're actually stored in computer memory
- C stores arrays in ***row-major order,*** with *row* 0 first, then *row* 1, and so forth
- How the m array is stored:



Multi-Dimensional Arrays

- Nested for loops are ideal for processing multidimensional arrays
- Identity matrix creation:

```
#define N 10
double ident[N][N];
int row, col;

for (row = 0; row < N; row++)
        for (col = 0; col < N; col++)
                if (row == col)
                        ident[row][col] = 1.0;
                else
                        ident[row][col] = 0.0;
```

Multi-Dimensional Arrays

Initialization

- We can create an initializer for a *two-dimensional* array by nesting one-dimensional initializers:

```
int m[5][9] = {{1, 1, 1, 1, 1, 0, 1, 1, 1},
               {0, 1, 0, 1, 0, 1, 0, 1, 0},
               {0, 1, 0, 1, 1, 0, 0, 1, 0},
               {1, 1, 0, 1, 0, 0, 0, 1, 0},
               {1, 1, 0, 1, 0, 0, 1, 1, 1}};
```

- Initializers for *higher-dimensional* arrays are constructed in a similar fashion
- C provides a variety of ways to abbreviate initializers for multidimensional arrays

Multi-Dimensional Arrays

Initialization

- We can even omit the inner braces:

```
int m[5][9] = {1, 1, 1, 1, 1, 0, 1, 1, 1,
               0, 1, 0, 1, 0, 1, 0, 1, 0,
               0, 1, 0, 1, 1, 0, 0, 1, 0,
               1, 1, 0, 1, 0, 0, 0, 1, 0,
               1, 1, 0, 1, 0, 0, 1, 1, 1};
```

- Once compiler has seen enough values to fill one row, it begins filling the next
- Omitting the inner braces can be risky, since an extra element (or even worse, a missing element) will affect the rest of the initializer

Multi-Dimensional Arrays

Initialization

- C99's designated initializers work with multidimensional arrays
- How to create 2 × 2 identity matrix:

```
double ident[2][2] = {[0][0] = 1.0, [1][1] = 1.0};
```

- As usual, all elements for which no value is specified will default to zero

Multi-Dimensional Arrays

Constant Arrays

- An array can be made *“constant”* by starting its declaration with the word const:

```
const char hex_chars[] = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9',
                          'A', 'B', 'C', 'D', 'E', 'F'};
```

- An array that’s been declared const should not be modified by the program
- Advantages of declaring an array to be const:
  - Documents that the program won’t change the array
  - Helps the compiler catch errors
- const isn’t limited to arrays, but it’s particularly useful in array declarations

Programming Task

Dealing a Hands of Cards

- The program illustrates both two-dimensional arrays and constant arrays
- The program deals a random hand from a standard deck of playing cards
- Each card in a standard deck has a ***suit*** (clubs, diamonds, hearts, or spades) and a *rank* (two, three, four, five, six, seven, eight, nine, ten, jack, queen, king, or ace)

Programming Task

Dealing a Hands of Cards

- The user will specify how many cards should be in the hand:

```
Enter number of cards in hand: 5

Your hand: 7c 2s 5d as 2h
```

- Problems to be solved:
  - How do we pick cards randomly from the deck?
  - How do we avoid picking the same card twice?

Variable-Length Arrays

- Length of array is computed during program execution

```
int m, n;

scanf("%d %d", &m, &n);

int arr[m][n];
```

- Can calculate exactly how many elements are needed

Variable-Length Arrays

- Advantage
  - Reduces wastage of memory
  - Avoids program failure
- Restrictions
  - Can’t have static storage
  - Can’t have initializers

Programming Task

Reverse a Series of Numbers (Revisited)

- Ask user how many numbers s/he wants to reverse

  How many numbers do you want to reverse? 5

  Enter 5 numbers: 5 2 1 5 3

  In reverse order: 3 5 1 2 5